Lista de Exercícios – Aula 12: Princípios SOLID com C#

Exercício 1 – Identificando Responsabilidades (SRP)

Analise a classe abaixo e identifique quantas responsabilidades ela possui. Depois, refatore aplicando o princípio da Responsabilidade Única.

```
public class Pedido
{
   public void SalvarPedido() { }
   public void EnviarEmailCliente() { }
   public void GerarNotaFiscal() { }
}
```

Objetivo: Separar responsabilidades em classes específicas.

Exercício 2 – Refatoração de Relatório (SRP)

Crie uma solução onde:

- Uma classe gere relatórios;
- Outra classe exporte PDF;
- Outra envie e-mail.

Evite que uma única classe faça tudo.

Objetivo: Aplicar separação de responsabilidades.

Exercício 3 – Sistema de Descontos (OCP)

Crie um sistema de descontos para:

- Cliente comum;
- Cliente VIP;
- Cliente Premium.

O sistema deve permitir adicionar novos tipos de desconto sem alterar o código existente.

Objetivo: Aplicar Aberto/Fechado usando interfaces.

Exercício 4 – Calculadora de Frete (OCP)

Implemente diferentes estratégias de frete:

- Sedex;
- PAC;
- Retirada na loja.

Evite usar muitos `if/else`.

Objetivo: Trabalhar extensibilidade com polimorfismo.

Exercício 5 – Violação de Liskov (LSP)

Analise o código:

```
public class Retangulo
{
  public virtual int Largura { get; set; }
  public virtual int Altura { get; set; }
}
```

Depois crie uma classe `Quadrado` herdando `Retangulo` e observe os problemas causados.

Objetivo: Entender violações do princípio de Liskov.

Exercício 6 – Modelagem Correta com LSP

Refatore o exercício anterior para evitar violações do LSP.

Dica: Avalie se herança é realmente necessária.

Exercício 7 – Interface Grande Demais (ISP)

A interface abaixo viola ISP:

```
public interface IFuncionario
{
  void Trabalhar();
  void Comer();
  void Dirigir();
}
```

Refatore criando interfaces menores e específicas.

Objetivo: Aplicar Segregação de Interface.

Exercício 8 – Impressora Multifuncional (ISP)

Modele:

- Impressora simples;
- Scanner;
- Impressora multifuncional.

Cada dispositivo deve implementar apenas o necessário.

Objetivo: Evitar métodos inúteis em classes.

Exercício 9 – Dependência Forte (DIP)

Analise o código:

```
public class PedidoService
{
   private EmailService _email = new EmailService();
}
```

Refatore usando abstrações e injeção de dependência.

Objetivo: Aplicar Inversão de Dependência.

Exercício 10 – Projeto Final SOLID

Crie um mini sistema de pedidos contendo:

- Cadastro de clientes;
- Processamento de pedidos;
- Envio de notificações;
- Geração de relatórios.

O projeto deve aplicar os 5 princípios SOLID.

Objetivo: Consolidar todos os conceitos da aula.

Desafio Extra

Escolha um projeto antigo seu (ou exemplo da internet) e:

1. Identifique violações de SOLID;
2. Explique os problemas encontrados;
3. Refatore aplicando boas práticas.